

Technical Environment Document Guide

Author: Raja SP, AWS Labs

Source: <https://github.com/awslabs/aidlc-workflows>

Adaptation: Ricardo de Luna Galdino, EngSoft Learn

Download: [technical-environment-guide.pdf](#)

Purpose

A Technical Environment Document defines the **technical tooling, standards, constraints, and preferences** that govern how a project is built. It is the technical counterpart to the Vision Document and serves as a binding reference during the Construction Phase of AI-DLC.

This document ensures that code generation, infrastructure design, and NFR decisions align with organizational standards, security policies, and team capabilities. Without it, AI-DLC stages will ask extensive clarifying questions to fill in these gaps, or worse, make assumptions that require rework.

When to Write a Technical Environment Document

- Before starting any new project (greenfield)
- Before modifying an existing project where technical constraints have changed (brownfield)
- When organizational technology standards have been updated
- When migrating between cloud providers, frameworks, or deployment models

Document Applicability

A Technical Environment Document can target one of two project contexts:

Context	Definition	Key Differences
Greenfield	No existing code. Building from scratch.	All choices are open. Document defines the starting point.
Brownfield	Existing codebase. Adding, modifying, or migrating.	Choices are constrained by what exists. Document defines what to keep, change, or avoid.

Structure your document for the applicable context. Sections below are marked with **(Greenfield)**, **(Brownfield)**, or **(Both)** to indicate where they apply.

Document Structure

1. Project Technical Summary (Both)

```
## Project Technical Summary

- **Project Name**: [Name]
- **Project Type**: [Greenfield / Brownfield]
- **Primary Runtime Environment**: [Cloud / On-Premises / Hybrid]
- **Cloud Provider**: [AWS / Azure / GCP / Multi-cloud / None]
- **Target Deployment Model**: [Serverless / Containers / VMs / Hybrid]
- **Team Size**: [Number of developers]
- **Team Experience**: [Key skills and experience levels relevant to tech choices]
```

2. Programming Languages (Both)

Define the languages the project must use, may use, and must not use.

```

## Programming Languages

### Required Languages
[Languages that must be used for specific purposes.]

| Language | Version | Purpose | Rationale |
|-----|-----|-----|-----|
| TypeScript | 5.x | Backend services, CDK infrastructure | Team expertise, type safety |
| Python | 3.12+ | Data processing, Lambda functions | ML library ecosystem |

### Permitted Languages
[Languages that may be used if justified, but are not required.]

| Language | Conditions for Use |
|-----|-----|
| Go | Approved for high-throughput microservices where latency is critical |
| Rust | Approved for systems-level components only with tech lead approval |

### Prohibited Languages
[Languages that must not be used, with reasoning.]

| Language | Reason |
|-----|-----|
| PHP | No team expertise, not aligned with platform direction |
| Ruby | Organizational standard prohibits new Ruby services |

```

Brownfield addition:

```

### Existing Language Inventory
[Languages currently in the codebase that must be maintained or migrated.]

| Language | Current Usage | Direction |
|-----|-----|-----|
| Java 11 | Core backend services | Maintain (upgrade to Java 21 in Phase 2) |
| JavaScript | Legacy frontend | Migrate to TypeScript |

```

3. Frameworks and Libraries (Both)

```
## Frameworks and Libraries

### Required Frameworks
[Frameworks that must be used for their respective domains.]

| Framework/Library | Version | Domain | Rationale |
|-----|-----|-----|-----|
| React | 18.x | Frontend UI | Organizational standard |
| Express | 4.x | API layer | Lightweight, team familiarity |
| AWS CDK | 2.x | Infrastructure as Code | AWS deployment target |
| Jest | 29.x | Unit testing | Consistent test runner across projects |

### Preferred Libraries
[Libraries that should be used when their capability is needed, but are not mandatory if the capability is not required.]

| Library | Purpose | Use When |
|-----|-----|-----|
| Zod | Runtime type validation | Any external data ingestion or API input |
| Pino | Structured logging | All services that emit logs |
| Axios | HTTP client | Outbound HTTP calls from services |

### Prohibited Libraries
[Libraries that must not be used. Include the preferred alternative.]

| Library | Reason | Alternative |
|-----|-----|-----|
| Moment.js | Deprecated, large bundle size | date-fns or Luxon |
| Lodash (full) | Bundle size | Native JS or lodash-es for specific imports |
| Request | Deprecated | Axios or native fetch |

### Library Approval Process
[How does a developer get approval to use a library not on the required or preferred lists?]

- [Describe approval process, e.g., "Submit a tech review request to the architecture team with justification, license check, and maintenance status assessment."]
```

4. Cloud Environment and Services (Both)

```
## Cloud Environment

### Cloud Provider
- **Primary Provider**: [AWS / Azure / GCP]
- **Account Structure**: [Single account / Multi-account / Organization]
- **Regions**: [Primary region(s) and disaster recovery region(s)]

### Service Allow List
[Services that are approved for use. Only services on this list may be used
without additional approval.]

| Service | Approved Use Cases | Constraints |
|-----|-----|-----|
| AWS Lambda | Event-driven compute, API handlers | Max 15 min timeout, 10GB memory |
| Amazon DynamoDB | Key-value and document storage | On-demand capacity for dev, provisioned
for prod |
| Amazon S3 | Object storage, static assets | Must enable versioning and encryption |
| Amazon SQS | Asynchronous message queuing | Standard queues preferred; FIFO only when orde
ring required |
| Amazon CloudWatch | Monitoring, logging, alarms | All services must emit structured logs |
| AWS Secrets Manager | Secrets storage | All credentials and API keys |
| Amazon API Gateway | REST and HTTP API exposure | HTTP APIs preferred over REST for new se
rvices |
| Amazon ECR | Container image registry | Required for all container-based services |
| AWS ECS Fargate | Container compute | Preferred over EC2-based ECS |
| Amazon RDS PostgreSQL | Relational data storage | Aurora Serverless v2 for variable worklo
ads |

### Service Disallow List
[Services that must not be used, with reasoning and approved alternatives.]

| Service | Reason | Alternative |
|-----|-----|-----|
| Amazon EC2 (direct) | Prefer managed/serverless compute | Lambda or ECS Fargate |
| Amazon ElastiCache | Cost and operational overhead for current scale | DynamoDB DAX or app
lication-level caching |
| AWS Elastic Beanstalk | Does not fit IaC workflow | CDK with ECS or Lambda |
| Amazon Kinesis | Complexity exceeds current needs | SQS or EventBridge |

### Service Approval Process
[How does a developer get approval to use a service not on the allow list?]

- [Describe process, e.g., "Submit a Cloud Service Request with business
justification, cost estimate, security review, and operational plan.
Requires architecture team approval."]
```

5. Preferred Technologies and Patterns (Both)

```
## Preferred Technologies and Patterns

### Architecture Patterns
| Pattern | When to Use | When Not to Use |
|-----|-----|-----|
| Serverless-first | Default for all new services | Workloads requiring persistent connections or >15 min processing |
| Event-driven | Asynchronous workflows, decoupled services | Simple CRUD with no downstream effects |
| Microservices | Independently deployable domains | Small projects with single-team ownership |
| Monolith (modular) | Single-team projects, early-stage MVPs | Multi-team or independently scalable domains |

### API Design Standards
- **Style**: [REST / GraphQL / gRPC] - [When to use each]
- **Versioning**: [URL path versioning (v1/v2) / Header-based]
- **Documentation**: [OpenAPI 3.x spec required for all REST APIs]
- **Naming Convention**: [kebab-case for URLs, camelCase for JSON fields]
- **Pagination**: [Cursor-based preferred, offset-based acceptable for admin APIs]
- **Error Format**: [Standard error response structure]

### Data Patterns
- **Primary Data Store**: [DynamoDB for service-owned data]
- **Relational Data**: [RDS PostgreSQL when relational queries are required]
- **Caching Strategy**: [Describe caching approach]
- **Data Ownership**: [Each service owns its data; no shared databases]

### Messaging and Events
- **Synchronous**: [HTTP/REST between services for request-response]
- **Asynchronous**: [SQS for task queuing, EventBridge for event distribution]
- **Event Schema**: [Describe event schema standards, e.g., CloudEvents format]

### Frontend Patterns (if applicable)
- **Component Library**: [e.g., Internal design system, Material UI, Shadcn]
- **State Management**: [e.g., React Context for local, Zustand for global]
- **Routing**: [e.g., React Router v6]
- **Build Tool**: [e.g., Vite]
```

6. Security Requirements (Both)

```
## Security Requirements

### Authentication and Authorization
- **Authentication Method**: [e.g., Amazon Cognito, OIDC, SAML]
- **Authorization Model**: [e.g., RBAC, ABAC, custom policy engine]
- **Token Format**: [e.g., JWT with RS256 signing]
- **Session Management**: [e.g., Token expiry, refresh token rotation]

### Data Protection
- **Encryption at Rest**: [Required for all data stores. Specify KMS key management.]
- **Encryption in Transit**: [TLS 1.2+ required for all communications]
- **PII Handling**: [Identify PII fields, masking requirements, retention policies]
- **Data Classification**: [Public / Internal / Confidential / Restricted]

### Network Security
- **VPC Requirements**: [Services that must run in VPC]
- **Security Groups**: [Least-privilege rules, no 0.0.0.0/0 ingress]
- **WAF**: [Required for all public-facing endpoints]
- **Private Endpoints**: [Use VPC endpoints for AWS service access where available]

### Secrets Management
- **Secrets Storage**: [AWS Secrets Manager / Parameter Store]
- **Rotation Policy**: [Automatic rotation every N days]
- **Access Policy**: [Least-privilege IAM policies per service]
- **Prohibited Practices**:
  - No secrets in source code, environment variables at build time, or config files
  - No shared credentials across services
  - No long-lived access keys

### Compliance Requirements
- **Standards**: [SOC 2, HIPAA, PCI-DSS, GDPR, FedRAMP, or "None specific"]
- **Audit Logging**: [All API calls logged, CloudTrail enabled, log retention period]
- **Vulnerability Scanning**: [Container image scanning, dependency scanning tools]

### Dependency Security
- **Dependency Scanning**: [Tool and frequency, e.g., Dependabot weekly, Snyk on PR]
- **License Policy**: [Allowed licenses: MIT, Apache 2.0, BSD. Prohibited: GPL, AGPL]
- **Update Policy**: [Critical vulnerabilities patched within N days]

### Security Compliance Framework

Every project must adopt a security risk framework and document how the project addresses each risk category in that framework. The choice of framework depends on the project's domain, regulatory environment, and organizational standards.

**Select one or more frameworks and document compliance per category:**

- **Framework chosen**: [Name and version, e.g., OWASP Top 10 (2021), NIST 800-53, CIS Controls v8, AWS Well-Architected Security Pillar,
```

```

SANS Top 25, or an internal organizational framework]
- Rationale: [Why this framework was selected. Reference regulatory
  requirements, customer contracts, or organizational policy if applicable.]

Common frameworks by context

| Context | Common Framework Choices |
|-----|-----|
| Web applications and APIs | OWASP Top 10, OWASP API Security Top 10 |
| Cloud-native infrastructure | AWS/Azure/GCP Well-Architected Security Pillar, CIS Benchmarks |
| Government / regulated | NIST 800-53, FedRAMP, ISO 27001 |
| General software | CIS Controls v8, SANS Top 25 |
| Internal / low-risk | Organizational security checklist (document it here) |

For each risk category in the chosen framework, document

1. How the project addresses it - Specific controls, patterns, and tooling that mitigate the risk
2. Not Applicable justifications - If a category does not apply, state why explicitly. Do not leave categories blank.
3. Deferred items - If a control is planned for a later phase, document the current gap and the target phase for remediation

Where to put the detailed compliance matrix

For small frameworks (10 or fewer categories), include the full matrix in this document under this heading.

For large frameworks (NIST 800-53, ISO 27001), create a separate file and reference it here:
- `security/[framework-name]-compliance.md`

See the CalcEngine example for a complete worked example using OWASP Top 10 (2021) as the chosen framework.

```


7. Testing Requirements (Both)

```
## Testing Requirements

### Test Strategy Overview
| Test Type | Required | Coverage Target | Tooling |
|-----|-----|-----|-----|
| Unit Tests | Yes | 80% line coverage minimum | Jest / pytest |
| Integration Tests | Yes | All service-to-service interactions | Jest + Testcontainers / py
test |
| End-to-End Tests | Conditional | Critical user journeys | Playwright / Cypress |
| Contract Tests | Conditional | All inter-service APIs | Pact |
| Performance Tests | Conditional | When SLA targets defined | k6 / Artillery |
| Security Tests | Yes | All public endpoints | OWASP ZAP / Snyk |

### Unit Testing Standards
- **Coverage Minimum**: [80% line coverage, 70% branch coverage]
- **Mocking Policy**: [Mock external dependencies, do not mock internal business logic]
- **Naming Convention**: [describe/it pattern, e.g., "describe('OrderService') > it('should
calculate total with tax')"]
- **Test Location**: [Co-located with source (e.g., `__tests__`) or separate tree (e.g., `t
ests/unit/`)]

### Integration Testing Standards
- **Scope**: [Test actual service interactions, database queries, and API contracts]
- **Environment**: [Local containers via Docker Compose / Testcontainers]
- **Data Management**: [Test fixtures, database seeding and cleanup approach]

### End-to-End Testing Standards
- **Scope**: [Critical user journeys only, not comprehensive UI testing]
- **Environment**: [Deployed staging environment]
- **Data-testid Requirements**: [All interactive elements must have stable data-testid attri
butes]

### Performance Testing Standards
- **Baseline Requirements**: [Define SLA targets: response time, throughput, error rate]
- **Test Scenarios**: [Load test, stress test, soak test]
- **Tooling**: [k6 / Artillery / JMeter]

### CI/CD Testing Gates
[Define which tests must pass at each stage of the pipeline.]

| Pipeline Stage | Required Tests | Failure Action |
|-----|-----|-----|
| Pre-commit | Linting, type checking | Block commit |
| Pull Request | Unit tests, integration tests | Block merge |
| Pre-deploy (staging) | E2E tests, contract tests | Block deploy |
| Post-deploy (production) | Smoke tests, health checks | Auto-rollback |
```

8. Example and Template Code Guidance (Both)

This section tells AI-DLC and the development team how to provide, use, and maintain example or template code that establishes project conventions.

Example and Template Code Guidance

Purpose of Example Code

Example code establishes the **canonical patterns** for the project. When AI-DLC generates code, it should follow these patterns rather than inventing new ones. When developers write code, they reference these examples for consistency.

When to Provide Example Code

Provide example or template code for any of the following:

- **Project structure setup** - Directory layout, file naming, module organization
- **API endpoint pattern** - How a standard endpoint is structured from route to response
- **Database access pattern** - How queries, transactions, and connections are handled
- **Error handling pattern** - Standard error types, error response format, logging
- **Authentication/authorization integration** - How auth is applied to endpoints
- **Testing pattern** - How a standard unit test and integration test are structured
- **Logging pattern** - Structured log format, what to log at each level
- **Configuration pattern** - How environment-specific configuration is loaded
- **Infrastructure as Code pattern** - How a standard CDK construct or Terraform module looks

How to Structure Example Code

Location

Store example code in a dedicated directory that AI-DLC and developers can reference:

...

```
project-root/
  examples/                                # Or "templates/" if preferred
    api-endpoint/
      handler.ts                          # Example API handler
      handler.test.ts                     # Corresponding test
      README.md                           # Explains the pattern and when to use it
    database-access/
      repository.ts                       # Example repository pattern
      repository.test.ts
      README.md
    infrastructure/
      standard-lambda-stack.ts            # Example CDK stack
      README.md
  ...
```

Structure of Each Example

Every example should include:

1. **Working code** - Not pseudocode. It must compile/run.
2. **Corresponding test** - Shows how to test the pattern.
3. **README.md** - Explains:
 - What pattern this demonstrates
 - When to use it
 - When NOT to use it
 - What to customize vs what to keep as-is
 - References to relevant standards from this Technical Environment Document

```

#### Example README Template

...

# [Pattern Name] Example

## What This Demonstrates
[One paragraph describing the pattern.]

## When to Use
- [Condition 1]
- [Condition 2]

## When Not to Use
- [Condition 1 - with alternative reference]

## File Inventory
| File           | Purpose           |
| ----- | ----- |
| handler.ts     | Example implementation |
| handler.test.ts | Test pattern      |

## Customization Guide
| Element           | Customize? | Notes           |
| ----- | ----- | ----- |
| Error handling structure | No          | Must follow project standard |
| Business logic       | Yes         | Replace with actual domain logic |
| Route path           | Yes         | Follow API naming conventions |
| Logging calls        | No          | Keep structured logging format |

## Related Standards
- [Link to API Design Standards section]
- [Link to Error Handling pattern]
...

### How AI-DLC Uses Example Code

During Code Generation, AI-DLC should:

1. **Read examples first** - Before generating any code, read relevant examples from the examples/ directory
2. **Follow established patterns** - Match the structure, naming, error handling, and testing patterns shown in examples
3. **Do not invent alternatives** - If an example exists for a pattern, use it. Do not create a different approach unless the example explicitly does not apply.
4. **Reference examples in plans** - Code Generation Plans should reference which examples apply to each step

### Maintaining Example Code

- **Update examples when standards change** - Examples must stay current with this Technical Environment Document
- **Review examples during onboarding** - New team members should read all examples before contributing code

```

- ****Version examples with the project**** - Examples live in the same repository and go through the same review process as production code
- ****Mark deprecated examples**** - If a pattern is superseded, rename the directory with a "deprecated-" prefix and add a note pointing to the replacement

9. Brownfield-Specific Sections

Include these sections only for brownfield projects.

```

## Brownfield: Existing Technical Inventory

### Current State Assessment
[Reference the Reverse Engineering artifacts if available, or provide
a summary of the current technical state.]

- **Current Languages**: [List with versions]
- **Current Frameworks**: [List with versions]
- **Current Infrastructure**: [Cloud services, deployment model]
- **Current Test Coverage**: [Percentage or qualitative assessment]
- **Known Technical Debt**: [Key items]

### Migration and Modernization Rules

#### What to Keep
[Technologies and patterns that should remain unchanged.]

| Technology | Reason to Keep |
|-----|-----|
| [Tech] | [Rationale] |

#### What to Migrate
[Technologies that should be replaced, with target and timeline.]

| Current | Target | Priority | Approach |
|-----|-----|-----|-----|
| JavaScript | TypeScript | High | Incremental file-by-file migration |
| REST API v1 | REST API v2 | Medium | New endpoints use v2, migrate existing in Phase 2 |

#### What to Remove
[Technologies, patterns, or dependencies that should be eliminated.]

| Item | Reason | Removal Timeline |
|-----|-----|-----|
| [Deprecated library] | [Security/maintenance concern] | [When] |

### Coexistence Rules
[When old and new patterns must coexist, define the rules.]

- **API versioning during migration**: [How v1 and v2 coexist]
- **Database schema migration**: [How schema changes are managed alongside existing data]
- **Feature flags**: [How new functionality is gated during transition]
- **Dependency conflicts**: [How conflicting library versions are managed]

```

How This Document Feeds Into AI-DLC

Technical Environment Section	AI-DLC Stage	How It Is Used
Project Technical Summary	Workspace Detection	Context for project classification
Programming Languages	Code Generation	Language selection and version constraints
Frameworks and Libraries	Code Generation, NFR Design	Dependency selection and prohibited library checks
Cloud Services Allow/Disallow Lists	Infrastructure Design	Service selection boundaries
Preferred Patterns	Application Design, Functional Design	Architecture and design pattern decisions
Security Requirements	NFR Requirements, NFR Design	Security pattern selection and compliance checks
Testing Requirements	Code Generation, Build and Test	Test strategy, tooling, and coverage targets
Example Code	Code Generation	Pattern reference during code generation
Brownfield Inventory	Reverse Engineering, Workflow Planning	Migration decisions and coexistence rules